One Size Does Fit All: Exploring Model Fusion for Software Engineering Tasks

YINGGANG QIU, National University of Defense Technology, China
YIHAO QIN, Information Support Force Engineering University, China
MINGYANG GENG, National University of Defense Technology, China
SHANGWEN WANG*, National University of Defense Technology, China
DEZUN DONG*, National University of Defense Technology, China

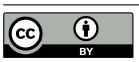
Large language models (LLMs) have achieved remarkable performance in software engineering (SE), and fine-tuning LLM for specific SE tasks has gradually become a new paradigm. However, storing fine-tuned checkpoints for multiple tasks incurs heavy storage and deployment complexity. Model fusion, which operates on fine-tuned parameters, offers excellent parameter compression and scalability, yet its effectiveness in the SE domain remains underexplored, making such an investigation essential for guiding the development of customized fusion techniques for the SE domain. To bridge this gap, we conduct a systematic study of model fusion in the SE contexts and reveal the following major findings: (1) when fusing programming languages (PLs) within the same task, model fusion usually works well and can enhance the performance of PLs with fewer data when PLs share similar features. (2) when fusing SE tasks of the same category within a same PL, all methods except TALL-Masks generally suffer substantial performance degradation on specific tasks; (3) when fusing SE tasks of different categories across different PLs, all existing model fusion methods exhibit significant performance degradation on certain tasks. In our evaluation results, TALL-Masks, which introduces a mask for each task to extract the most relevant dimensions from the fusion parameters, achieves promising performance. However, during parameters fusion, weak features (i.e., small variation in fine-tuned parameters) are easily overshadowed by strong ones (i.e., large variation in fine-tuned parameters) during parameter fusion, causing the constructed masks to fail to extract the most relevant parameters. To overcome this situation, we propose an improved version of TALL-Masks, called Scaling-Masks. The key idea is to amplify weak features to prevent them from being overshadowed by strong ones, which is achieved by scaling the value range of weak features to match that of strong features. Experimental results demonstrate that Scaling-Masks can significantly improve fusion performance for tasks with extremely weak features without affecting other tasks, with normalized accuracy improved by 63.49% for vulnerability detection when fusing SE tasks of different categories and 24.02% for PHP when fusing PLs in the code repair task.

CCS Concepts: • **Software and its engineering → Software maintenance tools**.

Additional Key Words and Phrases: Model fusion, Large language models, Program understanding tasks, Program generation tasks, Model merge

*Shangwen Wang and Dezun Dong are the corresponding authors.

Authors' Contact Information: Yinggang Qiu, National University of Defense Technology, College of Computer Science and Technology, Changsha, China, qyg@nudt.edu.cn; Yihao Qin, Information Support Force Engineering University, Wuhan, China, yihaoqin@qq.com; Mingyang Geng, National University of Defense Technology, College of Computer Science and Technology, Changsha, China, gengmingyang13@nudt.edu.cn; Shangwen Wang, National University of Defense Technology, College of Computer Science and Technology, Changsha, China, wangshangwen13@nudt.edu.cn; Dezun Dong, National University of Defense Technology, College of Computer Science and Technology, Changsha, China, dong@nudt.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE191
https://doi.org/10.1145/3808198

ACM Reference Format:
Yinggang Qiu, Yihao Qin, Mingyang Geng, Shangwen Wang, and Dezun Dong. 2026. One Size Does Fit All: Exploring Model Fusion for Software Engineering Tasks. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE191 (July 2026), 24 pages. https://doi.org/10.1145/3808198

1 Introduction

In recent years, large language models (LLMs) have developed rapidly [7, 63], and code-specific versions of LLMs [5, 39, 55] have also shown excellent performance in software engineering (SE) tasks [24], such as code summarization [19, 58], code search [10, 67] and fault localization [50–52]. Fine-tuning pre-trained LLMs for specific tasks has gradually become a prevailing paradigm [22, 82], as it not only leverages the knowledge of LLMs acquired from large-scale training data, but also enables customized adaptation for specific tasks.

When LLMs are required to handle multiple tasks, the simplest approach is to fine-tune a separate model for each setting, i.e., an SE task with a specific programming language (PL), and store them individually. However, this way significantly increases resource consumption during deployment, as it requires storing, managing, and loading a large number of models (considering that there are tens of different tasks during the software lifecycle [35] and tens of mainstream PLs). Such overhead poses serious challenges to the sustainable and green development of LLMs [57]. Consequently, the central goal of this paper is to explore **whether a single model can perform well across multiple SE tasks and PLs**. Traditional multi-task learning (MTL) enables LLMs to learn features from multiple tasks simultaneously, requiring only a single checkpoint to support multiple tasks. However, this approach also has several drawbacks in certain scenarios. First, when training data is unbalanced across tasks, tasks with fewer data are easily dominated by those with larger data, often leading to substantial performance degradation [17, 38, 41]. Second, MTL suffers from task interference (negative transfer) [32, 37, 46], where the optimization objectives or gradient directions of different tasks may conflict, making it difficult for the model to learn all tasks effectively. In addition, MTL also faces data privacy issues [4]. For example, in federated learning settings [45, 75], access to full training data for all tasks is not possible, and only model weights are available, making MTL infeasible. To intuitively understand the limitation of MTL under the common data imbalance scenario, we conduct experiments with CodeLlama-7B [55] on three program understanding tasks, including code search, vulnerability detection, and clone detection, where the training data for vulnerability detection is notably smaller than those for the other two tasks (experiment details will be introduced in Section 3). Figure 1 shows the performance of different methods on these tasks. It can be observed that, in the vulnerability detection task which has the least training data, the performance of the MTL model is significantly worse than the individually fine-tuned models.

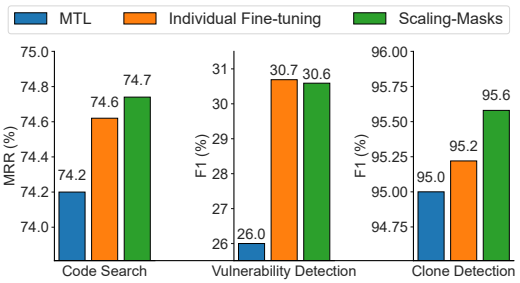


Fig. 1. The performance of different methods on three program understanding tasks.

As an emerging approach, model fusion, which is also called model merging in most work, has achieved significant success in integrating multi-task capabilities [73]. These techniques, which typically operate on the fine-tuned parameters of LLMs, can achieve multi-task capacity fusion without or with only a small amount of data, and provide significant parameter compression efficiency and excellent scalability. Although model fusion has achieved great success in the fields

of computer vision (CV) [56] and natural language processing (NLP) [23], its effectiveness in SE tasks remains largely unexplored. Little is known about how well a fused model would perform across diverse SE tasks. Such a study is both necessary and essential, as it can offer valuable insights for developing customized model fusion techniques within the SE domain. Therefore, to bridge this gap, we conduct a systematic empirical study on the effectiveness of model fusion techniques in SE tasks. Specifically, we investigate the following four research questions (RQs): (1) How do model fusion techniques perform when applied to multi-PLs fusion? (2) How do model fusion techniques perform when applied to multi-SE-task fusion? (3) How do model fusion techniques perform when applied to a rather challenging multi-SE-task multi-PL fusion scenario? and (4) How can we improve the performance of model fusion techniques in the SE contexts?

Our experiments are conducted on large-scale datasets (e.g., training data for each SE task exceeds 50k, with the largest dataset containing over 900k training samples), with CodeLlama-7B [55] as the backend LLM. We evaluate four state-of-the-art model fusion methods selected from three different categories, including *Basic methods*, *Subspace-based methods*, and *Weight-based methods*, according to a recent survey [73]. To answer the first RQ, we conduct empirical studies on one representative program understanding task (e.g., code search [66]) and one representative program generation task (e.g., code repair [40]), focusing on six PLs including Python, Java, JavaScript, Go, PHP, and Ruby. To answer the second RQ, we individually examine the performance of model fusion techniques on three program understanding tasks (code search, vulnerability detection, and clone detection) in the Java context and three program generation tasks (code generation, code repair, and code summarization) in the Python context. For the third RQ, we evaluate their effectiveness in fusing all six SE tasks to explore whether existing model fusion techniques can perform well in the challenging scenario of multi-task-multi-language environments. For the last RQ, we devise a new fusion strategy based on the observed findings from the previous RQs, and then re-evaluate its effectiveness under different settings.

Through an extensive evaluation, our study makes the following important findings:

- F1: For multi-PL fusion (MPLF), which involves fusing multiple programming languages within a single software engineering task, model fusion techniques generally maintain great performance. In addition, when different PLs share similar features, model fusion methods can enhance the performance of PLs with fewer data
- F2: When fusing either program understanding tasks or program generation tasks within the same PL individually, only the Subspace-based method, TALL-Masks [65], consistently maintains good performance across all tasks. Other model fusion methods generally experience substantial performance degradation on specific tasks.
- F3: When fusing program understanding tasks and program generation tasks together, where these two types of tasks are not only substantially different in nature but also require domain knowledge from different PLs, all existing model fusion methods exhibit significant performance degradation on certain tasks.

Through our in-depth analysis, we reveal that, during fusion, weak features (i.e., small variation in fine-tuned parameters) are easily overshadowed by strong features (i.e., large variation in fine-tuned parameters), and this is a universal phenomenon for leveraging existing model fusion techniques in the SE domain. This results in a significant performance drop for tasks with weak features after fusion. TALL-Masks, which introduces a mask for each task to extract the most relevant dimensions from the fusion parameters, achieves promising performance; however, when the difference in feature strength is substantially large, the constructed masks for tasks with extremely weak features fail to retain the most relevant dimensions. Based on this finding, we propose Scaling-Masks, an improved version of TALL-Masks. We are inspired by the **Feature Scaling** [31]

technique from the machine learning domain, where feature values are transformed into a common scale so that features with smaller numerical ranges are not overshadowed by those with larger magnitudes during training. Specifically, Scaling-Masks extends the TALL-Masks by adopting a normalization-like strategy that scales the range of weak features to that of the strong feature. This enhances the importance of the weak features and ensures they are better preserved in the fusion parameters, enabling the constructed masks to retain more task-relevant dimensions from the fusion parameters. As shown in Figure 1, our method maintains almost 100% of the original fine-tuned model's performance when being applied to fuse the three program understanding tasks.

To summarize, our contributions are as follows:

- **Empirical Analysis:** We provide a comprehensive empirical study on the application of model fusion techniques in the SE domain, encompassing (1) the fusion of multiple PLs within a single SE task, (2) the fusion of multiple SE tasks of the same PLs, and (3) the fusion of program understanding and program generation tasks across different PLs.
- **Improvement Strategy:** Based on the observed limitations of existing approaches, we propose a simple yet effective enhancement strategy (Scaling-Masks) that enhances weaker features to prevent them from being overshadowed by stronger ones. Our experiment results highlight the great potential of model fusion techniques in the SE domain, paving the way for more efficient and unified model management.
- **Open Source:** To facilitate reproducibility and foster future research, we release the implementation of our method along with the datasets used in our experiments.

2 Task Definition

Let there be N different tasks $(T_1, ..., T_N)$ and N corresponding models $(\Phi_{\Theta^{(1)}}, ..., \Phi_{\Theta^{(N)}})$, each obtained by fine-tuning the same pre-trained model $\Phi_{\Theta^{(0)}}$ on task T_i. The parameters of the model $\Phi_{\Theta^{(i)}}$ are denoted as $\Theta^{(i)} = \{\Theta_l^{(i)}\}_{l=1}^{L}$, where L is the number of the model's layers.

The goal of model fusion is to combine the set of parameters fine-tuned from multiple tasks into a single fixed parameter setting so that the fusion parameters can still support all tasks $(T_1, ..., T_N)$. Formally, let the fusion parameters be

$$\Theta_{\text{fusion}} = Fusion(\Theta^{(1)}, ..., \Theta^{(N)}) \tag{1}$$

During inference, Θ_{fusion} should be able to perform well on all the target tasks (i.e., maintaining performance comparable to those of the fine-tuned models).

3 Study Design

3.1 Model Fusion Techniques Selection

We adopt the Low-Rank Adaptation (LoRA) [25] to fine-tune the pre-trained model. LoRA introduces two low-rank trainable matrices $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$ on top of the frozen weights of the pre-trained model to approximate the incremental weights $\Delta W = BA$, thereby modifying the output from $y = Wx$ to $y = Wx + BAx$. This method retains the structure of the pre-trained weights and only optimizes a small number of additional parameters, which significantly reduces fine-tuning cost and is widely used for rapid adaptation of large-scale models. Previous studies have shown that LoRA can achieve performance comparable to full fine-tuning with fewer fine-tuning parameters [15, 28, 42, 82]. For N tasks $(T_1, \ldots, T_N)$ and their corresponding datasets $(D_1, \ldots, D_N)$, the LoRA parameters fine-tuned on corresponding datasets are denoted as $((A_1, B_1), \ldots, (A_N, B_N))$. In this work, the fusion objective is the fine-tuned LoRA parameters $((A_1, B_1), \ldots, (A_N, B_N))$.

Following the recent research [73], our study mainly investigates three mainstream categories of model fusion approaches: *Basic methods*, *Subspace-based methods*, and *Weight-based methods*. The

Table 1. Selected techniques in this study.

Type	Basic	Subspace-based	Weighted-based
Approach	Task Arithmetic [30]	Ties-Merging [72], TALL-Masks [65]	Learnable Fusion Coefficients [74, 77]

basic method is represented by Task Arithmetic, which serves as the basis for many subsequent approaches. Subspace-based methods project fine-tuned parameters into a subspace to reduce parameter conflicts or approximate the original fine-tuned parameters. Weight-based methods determine optimal fusion coefficients through some optimization strategies such as gradient descent. Table 1 summarizes the representative methods selected in this study. In the following, we briefly describe the working mechanisms of each method, and readers can refer to the original papers for more details.

Task Arithmetic [30]: The intuition behind this method is to treat the parameter variation of the fine-tuned model as task vectors, and achieve task fusion by performing simple arithmetic operations on these task vectors. In this study, we regard the fine-tuned LoRA parameters as task vectors $\tau_{\text{i}} = (A_i, B_i)$ and sum them to obtain the fusion parameters $\tau_{\text{fusion}} = \sum_{i=1}^{N} \tau_i$ [26, 78, 79]. Task Arithmetic further introduces a fusion coefficient to scale the fusion parameters. Specifically, it leverages validation sets or a small number of training sets to search for a specific fusion coefficient α, and then the inference parameters are given by $\hat{\tau} = \alpha \cdot \tau_{\text{fusion}}$.

Ties-Merging [72]: This work points out that traditional model fusion approaches often suffer from performance degradation due to parameter interference. This interference mainly arises from two sources: (a) redundant parameter values, i.e., most parameters undergo minimal variation during fine-tuning and contribute little to model performance, but can negatively affect others during merging; (b) sign conflicts, i.e., parameters at the same position may have inconsistent signs across different fine-tuned models. To address these issues, Ties-Merging (Trim, Elect Sign & Merge) proposes a three-step strategy to optimize the fusion process: (1) Trimming: Retain only the top-k parameters with the largest magnitude variation and discard the rest as redundant; (2) Sign Selection: For each parameter dimension, keep only the parameters with consistent signs across all models; (3) Disjoint Merging: Average the remaining non-zero parameters to compute the fusion parameters. In this work, we apply the above three operations to the fine-tuned LoRA parameters to obtain the fusion parameters. Similar to Task Arithmetic, Ties-merging also requires a fusion coefficient to scale the fusion parameters.

TALL-Masks [65]: This method introduces a specific mask for each task, which is used to extract the most relevant parameters for the task from the fusion parameters. Specifically, for each task T_i, a 1-bit mask m_i is constructed, where `true` represents retaining a parameter and `false` represents discarding it. For T_i, the objective is to minimize the $L1$ distance between the masked parameters and the task-specific fine-tuned parameters, that is:

$$m_i^* = \underset{m_i \in \{0,1\}^P}{\arg\min} \ \| \, m_i \odot p_{\text{fusion}} - p_i \, \|_1 \tag{2}$$

where p_{fusion} denotes the fusion parameters, p_i denotes the fine-tuned parameters of task T_i, and P denotes the number of all parameters. The mask m_i can be derived mathematically as:

$$m_i = \mathbf{1}\{|p_i| \geq |p_{\text{fusion}} - p_i|\} \tag{3}$$

The intuition of this formulation is that, for each parameter dimension, if the difference between the fusion parameter and the task-specific fine-tuned parameter (i.e., $|p_{\text{fusion}} - p_i|$) is too large, then this dimension of the fusion parameter is considered irrelevant to the task and should be discarded;

otherwise, it should be retained. During inference, the inference parameters for task T_i are given by $m_i \odot p_{\text{fusion}}$.

For the fusion parameters, the original strategy in TALL-Masks simply sums the task vectors, which we refer to *TALL-Masks-sum*. Moreover, we also consider an alternative strategy that is widely used in prior works [18, 27, 70] to obtain the fusion parameters: for each dimension of the fusion parameters, we retain the element with the largest absolute value from the fine-tuned parameters of each task as well as its original sign. We refer to this strategy as *TALL-Masks-max*.

Learnable Fusion Coefficients [74, 77]: Yang et al. [74] points out that using a single fusion coefficient for all task vectors may lead to suboptimal performance, while assigning separate coefficient for each task vector greatly enlarges the search space. For example, by assigning a fusion coefficient α_i to each task vector τ_i, the fusion parameters are $\tau_{\text{fusion}} = \sum_{i=1}^{N} \alpha_i \cdot \tau_i$. Assuming the search space for each coefficient is k, the total search space for N coefficients becomes k^N, meaning that the search space increases by a factor of k when adding one more task vector. To address this issue, recent studies have proposed adaptive methods for learning fusion coefficients [74, 77]. These methods define a loss function $\mathcal{L}(\cdot)$ to evaluate the performance of the fusion model and optimize the fusion coefficients through gradient descent. In this study, we assign fusion coefficients to each layer of the LoRA parameters and optimize them through gradient descent. Specifically, for the LoRA parameters $\{(A_i^l, B_i^l)\}_{i=1}^N$ in the l-th layer of the model across N tasks, we allocate $2N$ fusion coefficients $\{(\alpha_i^l, \beta_i^l)\}_{i=1}^N$. Thus, the fusion parameters for the l-th layer of the model are:

$$A_{\text{fusion}}^l = \sum_{i=1}^{N} \alpha_i^l A_i^l, \quad B_{\text{fusion}}^l = \sum_{i=1}^{N} \beta_i^l B_i^l \tag{4}$$

During inference, it directly uses the fusion parameters without other operations.

Since Task Arithmetic [30] and Ties-Merging [72] both require fusion coefficients to scale the parameters, Learnable Fusion Coefficient can be applied to both of these methods.

3.2 Software Engineering Tasks and Corresponding Datasets

This study involves the following six software engineering tasks:

Code Search: This task aims to retrieve relevant code snippets from a large corpus based on a natural language query, enabling developers to quickly locate and reuse code. We adopt the CodeSearchNet (CSN) [29] dataset as the benchmark for this task, which includes six PLs: Go, Java, JavaScript, PHP, Python, and Ruby.

Vulnerability Detection: This task aims to determine whether a program contains vulnerabilities, such as resource leaks or unsafe references. For this task, we use the dataset proposed by Zhang et al. [81] as the benchmark. This work constructs a multilingual benchmark dataset for vulnerability detection, covering three PLs: Python, Java, and JavaScript. Zhang et al. [81] extract Vulnerability-Fixing Commits (VFCs) from CVE entries in the National Vulnerability Database (NVD), and use the Tree-sitter tool to extract function-level source code from these commits.

Clone Detection: This task detects whether two code fragments are identical or highly similar, helping to reduce redundancy and improve software maintenance. For this task, we use BigCloneBench [59] as the benchmark. BigCloneBench is a widely used Java-based benchmark for clone detection, and has been further refined by Wang et al. [69].

Code Repair: This task automatically fixes bugs or errors in source code, aiming to improve software reliability and reduce manual debugging effort. For this task, we use the dataset from XCodeEval [36] as the benchmark. XCodeEval samples data from Codeforces and covers 11 mainstream PLs.

Code Generation: This task automatically generates source code based on a natural language description, helping developers speed up coding. Following previous work [71], we use the Self-OSS-Instruct-SC2-ExecFilter-50K [6] dataset as the fine-tuning dataset and adopt HumanEval [9] as the test dataset.

Code Summarization: This task generates concise natural language summaries for source code, making it easier for developers to quickly understand code functionality and purpose. For this task, we use the CSN-Python dataset from CodeXGLUE [44] as the benchmark.

3.3 Fine-tuning Details

In this study, we fine-tune **CodeLlama-7B** [55] by instruction tuning for each task. The prompt format follows the template proposed in Alpaca [60].

Code Search: Unlike Encoder-only models [16, 21], Decoder-only models restrict each token to attend only to its preceding tokens. To obtain a sentence-level embedding, we use the hidden state of the <eos> token from the final layer of CodeLlama as the representation of the entire input. Following prior work [8, 10], we fine-tune the model using InfoNCE [47], aiming to minimize the embedding similarity between a query and its corresponding code snippet while maximizing the similarity with other samples within the same batch.

Vulnerability Detection: Following prior work [34], we formulate the task as a binary classification problem, where the model is given a piece of code and asked to answer either "yes" (indicating the presence of a vulnerability) or "no" (indicating its absence). The fine-tuning objective is the standard causal language modeling loss, where the model is trained to predict the next token in a sequence given its preceding context.

Clone Detection: The fine-tuning strategy for this task follows a recent study, i.e., Cc2vec [13]. First, as in code search, for a given pair of code snippets, we obtain their sentence-level embeddings by taking the hidden state of the <eos> token from the final layer of CodeLlama. We then compute the cosine similarity between the two embeddings and normalize it as the predicted clone probability. Our objective is to maximize the predicted probability for clone pairs and minimize it for non-clone pairs. Therefore, we fine-tune the model using a binary cross-entropy loss. During inference, we consider two code snippets as clones if their cosine similarity is greater than 0.5.

Code Summarization, Code Repair and Code Generation: These tasks are standard generative tasks. Therefore, following previous work [42], the fine-tuning objective is the standard causal language modeling loss, where the model is trained to predict the next token given its context.

3.4 Evaluation Metrics

The metrics used for this work are as follows:

Code Search: We adopt *Mean Reciprocal Rank (MRR)* as the evaluation metric. MRR measures the average reciprocal rank of the first correct result in the candidate list, with higher values indicating that correct results appear earlier. This metric has been widely used in previous studies [20, 67, 68].

Clone Detection & Vulnerability Detection: We use *Accuracy*, *Precision*, *Recall*, and *F1-score* as metrics. These are commonly used to assess classification model performance [13, 49, 76].

Code Summarization: We adopt the *Bilingual Evaluation Understudy (BLEU)* [48] metric, which evaluates the n-gram overlap between the generated text and the reference text, and is commonly used to assess the grammatical similarity between them [19, 58]. In addition, we also use *BERTScore* [80] to measure the semantic similarity between generated and reference texts, which has been widely applied in summarization tasks [1, 58].

Code Generation & Code Repair: We use *pass@5* as the primary evaluation metric. It measures the probability that at least one of the top 5 generated candidates is correct. A higher score indicates better performance, and it has been adopted in several code generation studies [33, 54].

Table 2. Dataset statistics used in RQ1.

Task	Code Search			Code Repair	
	Train	Validation	Test	Train	Test
Python	412,178	23,107	22,176	461,356	710
Java	454,451	15,328	26,909	574,448	716
JavaScript	123,889	8,253	6,483	8,861	183
Go	317,832	14,242	14,291	13,085	294
Ruby	48,791	2,209	2,279	5,153	343
PHP	523,712	26,015	28,391	3,595	191

Table 3. Dataset statistics used in RQ2 and RQ3.

	Train	Validation	Test
Code Search	412,178	23,107	22,176
Vulnerability Detection	70,008	7,825	7,135
Clone Detection	901,058	415,416	415,416
Code Generation	50,661	\	164
Code Repair	461,356	\	710
Code Summarization	251,820	13,914	14,918

Model Fusion Metric: To evaluate the performance of model fusion methods, we use *Normalized Accuracy (N. Acc.)*, which is defined as

$$\text{N. Acc.} = \frac{S_{\text{fusion}}}{S_{\text{ft}}} \tag{5}$$

where S_{fusion} denotes the evaluation score of the fusion model and S_{ft} denotes the evaluation score of the fine-tuned model. This metric reflects how much of the original fine-tuned performance is retained after fusion, and has been widely adopted in the model fusion studies [30, 65, 72, 77].

3.5 Research Questions

Our study aims to answer the following questions.

RQ1: How do model fusion techniques perform when applied to multi-PL fusion (MPLF) within a same task? A software project often involves multiple PLs [11, 62], making MPLF essential. To answer this question, we conduct experiments on two representative SE tasks: code search, a typical program understanding task, and code repair, a typical program generation task. We fine-tune model on six mainstream PLs (Go, Java, JavaScript, PHP, Python, and Ruby), apply various model fusion methods to fuse the fine-tuned parameters, and evaluate the performance of the resulting fusion models.

RQ2: How do model fusion techniques perform when applied to multi-SE-task fusion (MSETF) within a same PL? To answer this question, we first investigate the performance of model fusion techniques on program understanding tasks in the Java context, including code search, vulnerability detection, and clone detection. Then, we examine model fusion techniques on fusing program generation tasks in the Python context, including code repair, code generation, and code summarization.

RQ3: How do model fusion techniques perform when applied to a rather challenging multi-SE-task multi-PL fusion (MSETMPLF) scenario? In this RQ, we explore a challenge scenario that whether model fusion techniques can effectively fuse both program understanding tasks and program generation tasks, which are not only substantially different in nature but also require domain knowledge from different PLs (Java vs. Python).

RQ4: How can we improve the performance of model fusion techniques for SE tasks? Based on the results and in-depth analysis from above RQs, we attempt to leverage the observed findings to improve the performance of model fusion techniques on SE tasks.

3.6 Data Usage and Experiment Settings

3.6.1 Data Usage. For **RQ1**, we use the sub-dataset of Python, Java, JavaScript, Go, PHP, and Ruby in the code search and code repair datasets introduced in Section 3.2. We use these datasets to fine-tune CodeLlama-7B, and then fuse the fine-tuned parameters of different PLs to evaluate the performance of model fusion techniques in MPLF. Table 2 presents the dataset statistics used in RQ1. For **RQ2** and **RQ3**, we use the Java subset for program understanding tasks (code search,

vulnerability detection, clone detection) and the Python subset for program generation tasks (code repair, code generation, code summarization). We use these datasets to fine-tune CodeLlama-7B, and then fuse the fine-tuned parameters of different SE tasks to evaluate the performance of model fusion techniques in MSETF and MSETMPLF. Table 3 presents the dataset statistics used in RQ2 and RQ3. For **RQ4**, we use the same datasets as those used in the corresponding fusion settings.

3.6.2 Experiment Settings. All experiments are conducted on 4 NVIDIA V100 GPUs. For all tasks, the fine-tuning learning rate is set to 8e-5 and the batch size is 64. The LoRA rank r is set to 32, with $\alpha = 64$. The LoRA layers are applied to the following modules: ["q_proj", "v_proj", "k_proj", "o_pro", "up_proj", "down_proj", "gate_proj"]. The goal of this study is to fuse multiple checkpoints and investigate how well the fused model preserves the domain-specific capabilities obtained under each setting. Therefore, we perform separate fine-tuning for each task–language pair to obtain individual checkpoints for subsequent fusion. The settings for each fusion method is as follow.

Task Arithmetic and Ties-Merging: Both methods require a grid search to find an appropriate scaling fusion coefficient α. For this purpose, we randomly sample 5,000 examples from the validation set for searching (or from the training set if the validation set is unavailable). For **RQ1**, since the optimal fusion coefficients for each task are mostly consistent, we use the same scaling coefficient α for all tasks. For **RQ2** and **RQ3**, the optimal fusion coefficients vary significantly across tasks, so we search for a separate coefficient for each task individually. For Task Arithmetic, the search space is $[0.1, 1.0]$ with a step size of 0.1. For Ties-Merging, we retain the top 70% of parameters by magnitude, and the search space is $[0.5, 2.5]$ with a step size of 0.1.

Learnable Fusion Coefficients: For this method, we randomly sample 5,000 examples from the validation set for searching (or from the training set if the validation set is unavailable) of each task for training, which is significantly smaller than the full training dataset. The training objective remains the same as the loss function used during fine-tuning. For **RQ1**, the fusion coefficients are initialized with the optimal values obtained by grid search. The learning rate is set to 0.001, and training is performed for 3 epochs. For **RQ2** and **RQ3**, the fusion coefficients are initialized as the average of the optimal coefficients for each individual task obtained by grid search. The learning rate is set to 0.003, and training is performed for 5 epochs.

TALL-Masks: Since this method does not involve additional search or training, no additional settings are required.

4 Study Result

4.1 RQ1: The Performance of Model Fusion in Multi-PL Fusion within a same SE task

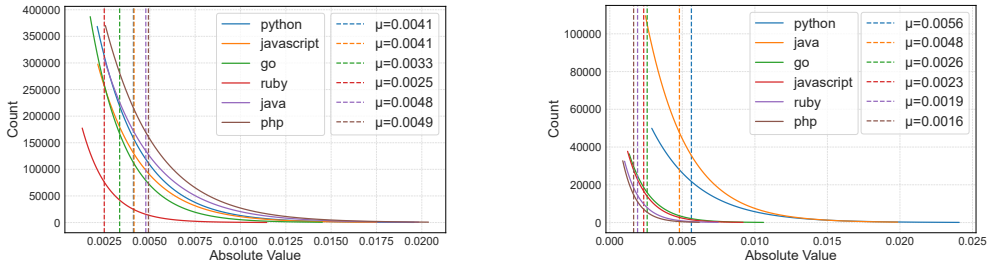
Table 4 presents the results of MPLF in code search and code repair tasks under different model fusion methods. The values in parentheses represent the Normalized Accuracy. We observe that *TALL-Masks-sum* obtains very poor performance while *TALL-Masks-max* achieves promising performance. A similar phenomenon can also be observed in MSETF and MSETMPLF. This indicates that max fusion is more effective than sum fusion to obtain fusion parameters for *TALL-Masks* in the SE domain. Therefore, in the following sections, we only report the results of *TALL-Masks-max* and refer to it as *TALL-Masks* by default.

In the code search task, for all PLs, different fusion methods consistently retain over 96% of the performance of the original fine-tuned models. Among them, *TALL-Masks* achieves the best performance and even slightly improves compared to the original fine-tuned models. In the code repair task, for JavaScript and Ruby languages, all fusion methods maintain excellent performance. However, for other languages, some fusion methods show significant performance drops. In the PHP language, almost all methods show a substantial decrease. For example, compared with the fine-tuned model, the normalized accuracy of the *Task Arithmetic*, *Ties*, and *TALL-Masks* drop by

Table 4. The performance of different model fusion methods in multi-programming-language fusion

Task		Code Search							Code Repair						
		MRR (N. Acc.)							pass@5 (N. Acc.)						
Method	Learnable Coefficients	Python	Java	JavaScript	Go	Ruby	PHP	Avg.	Python	Java	JavaScript	Go	Ruby	PHP	Avg.
Fine-tuning	/	84.58	74.62	76.81	76.70	82.02	75.55	78.38	24.61	29.98	23.34	29.41	17.82	34.31	26.58
Task Arithmetic	no	82.68 (97.75)	74.84 (100.29)	77.74 (101.22)	75.19 (98.04)	81.57 (99.45)	72.41 (95.85)	77.41 (98.77)	21.72 (88.29)	27.18 (90.67)	27.28 (116.88)	24.89 (84.61)	20.08 (112.67)	27.22 (79.34)	24.73 (95.41)
	yes	82.10 (97.07)	74.67 (100.07)	77.69 (101.17)	74.69 (97.39)	81.45 (99.90)	72.82 (96.39)	77.24 (98.56)	23.08 (93.81)	28.17 (93.96)	**30.33 (129.94)**	26.17 (88.99)	**22.66 (127.12)**	**31.25 (91.09)**	**26.94 (104.15)**
Ties-Merging	no	82.40 (97.41)	74.99 (100.49)	77.56 (100.99)	73.67 (96.06)	80.96 (98.71)	73.79 (97.67)	77.23 (98.55)	23.77 (96.60)	29.20 (97.39)	26.69 (114.39)	17.3 (58.82)	20.02 (112.33)	20.94 (61.04)	22.99 (90.10)
	yes	82.52 (97.56)	75.08 (100.61)	**77.85 (101.37)**	73.83 (96.26)	81.54 (99.41)	73.74 (97.61)	77.43 (98.81)	23.88 (97.07)	28.71 (95.77)	26.99 (115.64)	19.75 (67.14)	20.83 (116.87)	23.08 (67.26)	23.87 (93.29)
TALL-Masks -max	/	**84.61 (100.03)**	**75.23 (100.84)**	77.48 (101.01)	**76.67 (99.97)**	**81.93 (99.88)**	**75.93 (100.49)**	**78.64 (100.37)**	**24.4 (99.14)**	**31.18 (103.99)**	23.33 (99.97)	**29.29 (99.56)**	17.71 (99.37)	26.69 (77.79)	25.43 (96.62)
TALL-Masks -sum	/	3.39 (4.01)	0.89 (1.19)	1.82 (2.37)	0.48 (0.63)	3.65 (4.45)	0.41 (0.54)	1.77 (2.20)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)

† A deeper red indicates a lower Normalized Accuracy.



(a) Code search task (b) Code repair task

Fig. 2. Distribution of the absolute values of the fine-tuned parameters across different PLs.

20.66%, 38.96%, and 22.21%, respectively. In the Go language, for the *Ties*, there is a significant drop regardless of whether *Learnable Fusion Coefficients* is used, with normalized accuracy decreasing by 41.18% and 32.86%, respectively.

From the above results, we can observe that some PLs experience a larger performance drop in the code repair task than in the code search task. We attribute it to the more unbalanced data distribution across different PLs in the code repair task (as shown in Table 2). Figure 2 presents the distribution of the absolute values of fine-tuned parameters (note that for LoRA, the absolute values of the fine-tuned parameters are equivalent to the extent of its variation) for different PLs in both code search and code repair tasks. Since the parameter distribution of the LoRA A matrix shows minimal difference across different PLs, we only present the parameter distribution of the LoRA B matrix, where larger values indicate more important parameters while smaller ones are less significant (we present the top-50% largest parameters in the figure). We can see that PLs with fewer training data samples tend to have smaller parameter variation, and the variation across PLs is generally larger in code repair than in code search. In particular, the parameter variation for PHP in code repair is the smallest, which causes the significant performance drops of all fusion methods.

Finding-1. *In MPLF, almost all model fusion methods achieve competitive performance across all languages in the code search task. However, in the code repair task, different fusion methods would suffer performance drops on PLs with smaller parameter variation.*

Unlike the findings in NLP and CV domains, where *Ties* and *Learnable Fusion Coefficients* typically outperform *Task Arithmetic*, we observe no such advantage in MPLF for code-related tasks. For

example, *Ties* achieves a 3.5% improvement in normalized accuracy over *Task Arithmetic* on NLP tasks with the T0-3B model [72]. However, in our study, there is a performance drop of 0.22% and 5.31% in normalized accuracy from *Task Arithmetic* to *Ties* on code search and code repair tasks, respectively. From the previous study [77], *Learnable Fusion Coefficients* improve performance by 9.15% in normalized accuracy over *Task Arithmetic* on CV tasks using the ViT-L/14 model. However, in MPLF for SE tasks, *Learnable Fusion Coefficients* show limited improvements, with gains in the code repair task (e.g., 8.74% improvement in normalized accuracy over *Task Arithmetic*) but no clear benefit and even a slight drop in the code search task (e.g., 0.21% degradation in normalized accuracy over *Task Arithmetic*).

Finding-2. *In MPLF, Ties and Learnable Fusion Coefficients cannot generally improve fusion performance like what they have achieved in NLP and CV tasks. Ties shows no improvement in MPLF, while Learnable Fusion Coefficients enhances performance in the code repair task but brings no clear benefit to the code search task.*

```
// The Code of JavaScript
function main() {
  //...

  const weight = readline();
- // buggy
- const output = weight % 2 === 0 ? "YES" : "NO"

+ // fixed after fused model
+ const output = (weight % 2 === 0 && weight > 2)
      ? "YES" : "NO"
  //...
}
```

```
// The Code of Go
func main() {
  //...
  fmt.Scan(&a)
- // buggy
- if a % 2 == 0 {
+ // fixed by fine-tuned models
+ if a % 2 == 0 && a > 2 {
   fmt.Println("YES")
  } else {
    fmt.Println("NO")
  }
}
```

Fig. 3. A successful fix case after fusion in the code repair task.

In both tasks, we observe that almost all methods show improvements on JavaScript, with higher improvements in the code repair task than in code search (e.g., compared to the fine-tuned model, *Task Arithmetic* yields gains of 0.97% and 3.94% on JavaScript for the two tasks, respectively). We notice that the training data for JavaScript is relatively fewer compared to other languages (referred to in Table 2). A similar observation can also be made for the Ruby language in the code repair task. The results suggest that PLs with fewer training data may learn from other PLs during fusion.

Figure 3 presents a case from the code repair task. In this example, the fine-tuned model on the JavaScript language fails to correctly repair a buggy condition (`weight % 2 == 0`), while the fine-tuned model on the Go language successfully repairs a similar bug (`a % 2 == 0`) for the same problem. Surprisingly, after model fusion, the fused model leverages the repair patterns learned from Go to correctly fix the JavaScript bug which is previously failed to be handled by the fine-tuned model. This case demonstrates that when datasets across different PLs share similar features, model fusion can enhance performance by transferring learned features from other languages.

Finding-3. *In MPLF, when sharing similar features with other languages, the performance on PLs with fewer training data can be maintained or even improved during model fusion.*

4.2 RQ2: The Performance of Model Fusion in Multi-SE-task Fusion within a Same PL

Table 5 presents the fusion results for program understanding tasks. In contrast to MPLF, the correlations across different SE tasks are usually weaker, which results in stronger conflicts among

Table 5. The performance of different model fusion methods in program understanding tasks fusion

Task		Code Search	Vulnerability Detection				Clone Detection			
Method	Learnable Coefficients	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)
Fine-tuning	/	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22
Task Arithmetic	no	68.87 (92.29)	4.14	16.18	**46.45**	24.00 (80.00)	98.43	96.88	91.52	94.12 (98.72)
	yes	68.34 (91.58)	90.84	35.53	26.11	30.11 (98.11)	98.69	**97.79**	92.57	95.11 (99.88)
Ties-Merging	no	69.71 (93.41)	0.00*	0.00	0.00	0.00 (0.00)	98.62	95.17	**94.73**	94.95 (99.71)
	yes	64.06 (85.85)	90.75	19.61	5.60	8.71 (28.38)	98.17	97.41	89.01	93.02 (97.68)
TALL-Masks	/	**74.75 (100.17)**	**92.55**	**51.06**	22.39	**31.12 (101.40)**	**98.71**	96.47	94.01	**95.23 (100.01)**

* Because the fusion model fails to follow the instructions and neither answered "yes" nor "no", the accuracy is 0.00.

Table 6. The performance of different model fusion methods in program generation tasks fusion

Task		Code Repair	Code Sum.		Code Gen.
Method	Learnable Coefficients	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	/	24.61	8.13	88.66	67.00
Task Arithmetic	no	17.55 (71.31)	8.76 (107.75)	88.72 (100.07)	61.70 (92.90)
	yes	18.89 (76.75)	8.86 (108.98)	88.90 (100.07)	53.60 (80.00)
Ties-Merging	no	10.54 (42.82)	**9.65 (118.69)**	**89.13 (100.53)**	57.70 (86.12)
	yes	17.09 (69.44)	9.05 (113.16)	88.92 (100.29)	55.80 (83.28)
TALL-Masks	/	**23.59 (95.86)**	8.30 (102.09)	88.72 (100.07)	**66.80 (99.70)**

Code Sum.: Code Summarization; **Code Gen.:** Code Generation

the tasks. Except for *TALL-Masks*, which can retain nearly 100% of the original performance across all tasks, other fusion methods perform differently across different tasks. Specifically, on vulnerability detection, both *Task Arithmetic* and *Ties* show almost a complete drop in accuracy. *Ties* even loses its instruction-following capability (i.e., neither answering "yes" nor "no"), which may be caused by the destruction of the capability learned during fine-tuning in the fusion process. Although the *Learnable Fusion Coefficients* method can recover the accuracy of vulnerability detection, it still suffers notable decreases in precision and recall; for instance, compared with the original fine-tuned model, *Task Arithmetic* with *Learnable Fusion Coefficients* incurs a 14.68% drop in precision, while *Ties* exhibits 30.60% and 16.50% reductions in precision and recall, respectively. On code search, we noticed that both *Task Arithmetic* and *Ties* suffer from a moderate performance loss, with *Learnable Fusion Coefficients* even further harming the model performance. For example, when using *Task Arithmetic*, the MRR metric decreases from 74.62% to 68.87%, and further drops to 68.34% after applying *Learnable Fusion Coefficients*. When it comes to clone detection, we found that both fusion methods show almost no performance drop, maintaining more than 97% performance of the fine-tuned model.

Table 6 presents the fusion results for program generation tasks. Similar to the program understanding task, *TALL-Masks* maintains nearly 100% performance across all tasks, while other methods experience significant drops in some tasks. Specifically, except for *TALL-Masks*, there are considerable performance declines in both code repair and code generation tasks, whereas the code summarization task is almost unaffected or even slightly improved. Furthermore, the impact of *Learnable Fusion Coefficients* on fusion tasks is unstable: it provides some benefit for the code repair task, shows almost no effect on the code summarization task, and even has a negative impact on the code generation task, where *Task Arithmetic* and *Ties* decrease by 6.40% and 1.90%, respectively. This could be incurred by conflicts between these tasks, such as gradient conflicts.

From the above results, we observe that model fusion performs well on some tasks (e.g., clone detection, code summarization), while causing significant performance degradation on other tasks. To explore the underlying reasons, we conduct the following analysis. Figure 4 shows the distribution of the absolute values of the fine-tuned parameters across different tasks. It can be observed that

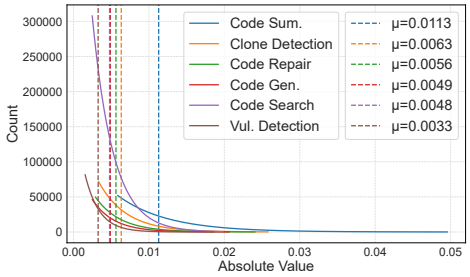


Fig. 4. Distribution of the absolute values of the fine-tuned parameters across different tasks.

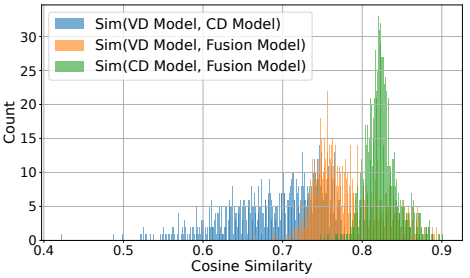


Fig. 5. Cosine similarities between the embeddings of different models.

Table 7. The performance of different model fusion methods in all SE tasks fusion

Task		Code Search	Vulnerability Detection				Clone Detection				Code Repair	Code Sum.		Code Gen.
Method	Learnable Coefficients	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	/	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22	24.61	8.13	88.66	67.00
Task Arithmetic	no	68.06 (91.21)	78.41	17.31	49.62	**25.67 (83.64)**	94.93	**99.75**	63.09	77.29 (81.16)	15.38 (62.49)	1.11 (13.65)	80.85 (91.19)	60.30 (90.00)
	yes	62.24 (83.41)	**91.46**	28.15	8.95	13.67 (44.54)	98.42	99.14	89.23	93.92 (98.63)	15.08 (61.27)	**8.56 (105.29)**	**88.93 (100.30)**	53.90 (80.45)
Ties-Merging	no	66.68 (89.36)	26.52	9.27	**83.39**	16.70 (54.41)	94.99	97.27	65.19	78.06 (81.98)	10.91 (44.33)	1.25 (15.37)	81.76 (92.22)	56.40 (84.18)
	yes	66.57 (89.21)	91.17	9.52	1.11	2.03 (6.61)	89.98	76.65	38.47	51.23 (53.80)	13.86 (56.31)	6.98 (84.75)	88.38 (99.68)	46.40 (69.25)
TALL-Masks	/	**73.78 (98.87)**	58.24	**77.27**	6.34	11.72 (36.72)	**98.81**	97.17	**94.02**	**95.57 (100.37)**	**24.23 (98.45)**	**8.56 (105.29)**	88.75 (100.10)	**68.10 (101.64)**

some tasks have large parameter variation, which we refer to as strong features, while others have small parameter variation, which we refer to as weak features. Through the results from Table 5 and Table 6, we can observe that tasks with stronger features experience almost no degradation after fusion (e.g., clone detection and code summarization) , while tasks with weaker features (e.g., vulnerability detection and code repair) often suffer significant drops. We hypothesize that, in MSETF, tasks with stronger features dominate after fusion, suppressing tasks with weaker features.

To further verify the above hypothesis, we randomly sampled 1,000 examples from the test set of the vulnerability detection task. For each sample, we obtained the hidden state of each input token from different models and averaged them as its embedding representation. Figure 5 presents the histogram of cosine similarities between the embeddings of the vulnerability detection fine-tuned model (VD Model), the clone detection fine-tuned model (CD Model) and the Fusion Model (using *Task Arithmetic* without *Learnable Fusion Coefficients*). In the figure, it can be observed that the cosine similarity of the embeddings between the VD Model and the CD Model is the lowest, indicating a clear distinction between the features of the two tasks. However, the Fusion Model embeddings are much more similar to those of the CD Model than to the VD Model. This suggests the Fusion Model's features are dominated by clone detection, masking vulnerability detection's weak features and ultimately leading to a significant performance drop.

Finding-4. *In MSETF, except for TALL-Masks, other fusion approaches tend to be dominated by tasks with stronger features when fusing program understanding or program generation tasks individually. These dominant tasks can retain great performance, while tasks with weaker features have a significant performance drop.*

4.3 RQ3: The Performance of Model Fusion in Multi-SE-task Multi-PL Fusion

Table 7 presents the results of unifying the program understanding tasks and the program generation tasks through different model fusion methods. In such a challenge scenario of fusing two distinct categories across different PLs, we find that all fusion methods would show substantial degradation in certain SE tasks. For *Task Arithmetic*, *Ties*, and *Learnable Fusion Coefficients*, it can be observed that when fusing two substantially different task categories, there is a significant performance drop across almost all tasks. Although *TALL-Masks* maintains nearly 100% performance when fusing program understanding and generation tasks separately, its performance drops significantly in the vulnerability detection task when the two categories are fused.

Finding-5. *When unifying tasks of two distinct categories across different PLs, almost all fusion methods experience significant performance drops on certain tasks. Particularly, TALL-Masks maintain nearly 100% ability of the fine-tuned model on all tasks except vulnerability detection.*

To understand why *TALL-Masks* fails to preserve the ability of the fine-tuned model for the vulnerability Detection task in MSETMPLF, we show the parameter retention ratios (i.e., the ratio of parameters preserved by the constructed mask to the total number of parameters) of *TALL-Masks* across different tasks in Figure 6. We observed that the vulnerability detection task has the lowest parameter retention rate, and its retention rate for the most important parameters (i.e., parameters in the top-10% largest absolute values) is significantly lower than that of other tasks. It means that when computing the fusion parameters, the weak features of these tasks can be overshadowed by stronger features (i.e., larger parameter variation) from other tasks, which causes the constructed masks to be biased, resulting in the loss of some key features. It is worth noting that this observation (i.e., for *TALL-Masks*, lower retention ratios of parameters may cause worse model performance) holds not only for MSETMPLF, as we also find a similar trend for MPLF and MSETF. For example, the PHP language of code repair in MPLF has the lowest parameter retention rate, which also suffers from a large performance degradation (refer to Table 4). Due to space constraints, all data is provided in our online repository.

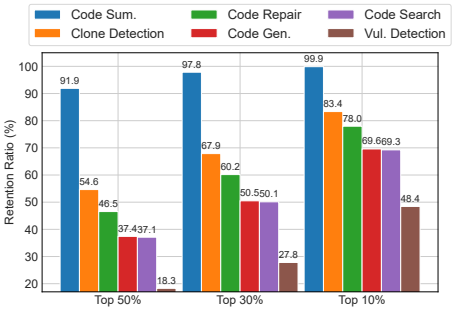


Fig. 6. Retention ratios of parameters in different Top-k largest absolute values using TALL-Masks in MSETMPLF.

Finding-6. *For TALL-Masks, weak features are easily overshadowed by strong features in the fusion parameters, resulting in the constructed masks failing to effectively retain important parameters.*

4.4 RQ4: Improvements of Model Fusion Techniques for SE Tasks

Through our investigation from RQ1 to RQ3, we have identified a prevalent issue in current model fusion techniques where strong features tend to dominate over weaker ones during fusion. Building upon this insight, we propose to enhance *TALL-Masks* as it is the leading fusion approach.

Method. The methodology of our enhanced iteration, *Scaling-Masks*, is depicted in Figure 7. Notably, in RQ1 and RQ3, we have noticed a notable performance decline for tasks with the weakest features in *TALL-Masks*. We attribute this phenomenon to the *Direct Max Fusion* strategy (illustrated at the left of Figure 7), when obtaining the fusion parameters, weak features tend to be overwhelmed by strong features. As a result, during mask construction, the large gap between

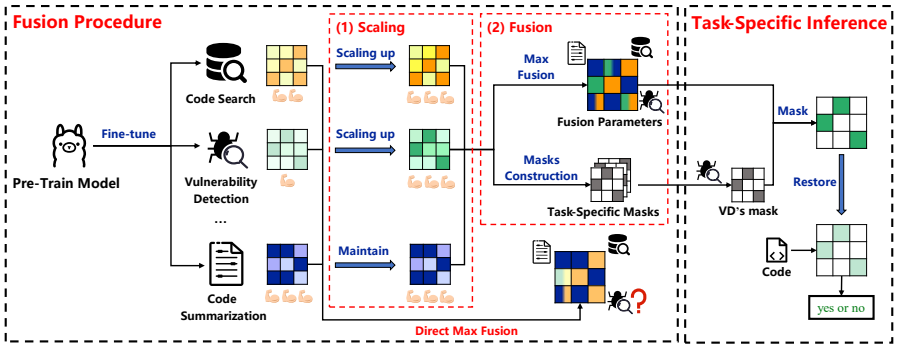


Fig. 7. The Workflow of the Scaling-Masks.

fine-tuned and fusion parameters may cause many parameters to be masked, which unexpectedly discards many weak yet task-relevant features. To intuitively understand the above mechanism, we provide a simplified example as follows. Let $\mathbf{v_s} = [0.75, \ldots, 1.23]$ and $\mathbf{v_w} = [0.09, \ldots, 0.13]$ be the two task vectors with strong features and weak features, respectively. After the Max fusion, the new model would only retain the max value on each dimension, which results in the fused task vector $\mathbf{v_f} = [0.75, \ldots, 1.23]$. In this case, *TALL-Masks* would generate two masks $\mathbf{m_s} = [1, \ldots, 1]$ and $\mathbf{m_w} = [0, \ldots, 0]$ for each task using the formula mentioned in Section 3.1. It is obvious that since the weak features have not been reserved in the fused task vector $\mathbf{v_f}$, the mask $\mathbf{m_w}$ has to drop all the useful parameters, which leads to the degradation of the task with weak features.

To address the above issue, we propose **Scaling-Masks**, which is inspired by data normalization and Feature Scaling techniques [3, 31] that map features with different ranges to a common scale, thus preventing features of high value from suppressing features of low value [2, 43]. *Scaling-Masks* mainly consists of two stages: the *Scaling* stage and the *Fusion* stage. In the *Scaling* stage, we map features of different strengths into the same strength. Specifically, we use s_i to denote the feature strength of task T_i, where s_i is calculated as the mean of the absolute values of the fine-tuned parameters. This method has also been used in a prior study [27] to evaluate the gap between fused and fine-tuned models. Next, we choose the task with the largest feature strength as the pivot task T_{pivot}, whose strength remains unchanged during scaling. For each task T_i, we scale its feature strength to match that of T_{pivot} by multiplying its parameters with a scaling factor $r_i = s_{\text{pivot}}/s_i$. This unifies all features to the same strength, ensuring that weak features would not be overwhelmed during the parameter fusion process. In the *Fusion* stage, we adopt the same procedure of *TALL-Masks* introduced in Section 3.1. Since both weak and strong features are preserved after applying the scaling operation, the mask construction is now more likely to identify the task-relevant parameters. Recall the aforementioned example, we regard task T_s with vector $\mathbf{v_s}$ as the pivot task, where $s_{\text{pivot}} = 0.99$. Then, for the task T_w with vector $\mathbf{v_w}$, we can acquire the scaling factor r = 0.99/0.11 = 9. In this case, the scaled vector for task T_w is $\hat{\mathbf{v_w}} = [0.81, \ldots, 1.17]$. We can observe that the new fused vector using *Direct Max Fusion* is currently $\hat{\mathbf{v_f}} = [0.81, \ldots, 1.23]$, which simultaneously retains parameters from tasks with strong features and weak features (i.e., value 0.81 from $\hat{\mathbf{v_w}}$ and 1.23 from $\mathbf{v_s}$).

As shown in the right part of Figure 7, during the inference procedure, we first use the corresponding mask for the given task T_i to extract the most relevant parameters from the fusion parameters. Note that the strength of the parameters is still aligned with the pivot task, which is different from the strength of the task before fusion. Consequently, we further rescale the extracted parameters back to the original task's strength.

Table 8. The performance of Scaling-Masks in multi-programming language fusion

Task	Code Search							Code Repair						
	MRR (N. Acc.)							pass@5 (N. Acc.)						
Method	Python	Java	JavaScript	Go	Ruby	PHP	Avg.	Python	Java	JavaScript	Go	Ruby	PHP	Avg.
Fine-tuning	84.58	74.62	76.81	76.70	82.02	75.55	78.38	24.61	29.98	23.34	29.41	17.82	34.31	26.58
TALL-Masks	84.61 (100.03)	75.23 (100.84)	**77.48 (101.01)**	76.67 (99.97)	81.93 (99.88)	75.93 (100.49)	78.64 (100.37)	24.4 (99.14)	31.18 (103.99)	23.33 (99.97)	29.29 (99.56)	17.71 (99.37)	26.69 (77.79)	25.43 (96.62)
Scaling-Masks	**84.63 (100.05)**	**75.28 (100.88)**	**77.48 (101.01)**	**76.74 (100.05)**	**82.09 (100.08)**	**75.95 (100.53)**	**78.70 (100.41)**	**25.02 (101.67)**	**31.57 (105.30)**	**24.36 (104.37)**	**30.71 (104.42)**	**18.77 (105.33)**	**34.93 (101.81)**	**27.56 (103.81)**

Table 9. The performance of Scaling-Masks in all SE tasks fusion

Task	Code Search	Vulnerability Detection				Clone Detection				Code Repair	Code Sum.		Code Gen.
Method	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22	24.61	8.13	88.66	67.00
TALL-Masks	73.78 (98.87)	58.24	**77.27**	6.34	11.72 (36.72)	**98.81**	**97.17**	94.02	**95.57 (100.37)**	24.23 (98.45)	8.56 (105.29)	88.75 (100.10)	**68.10 (101.64)**
Scaling-Masks	**74.63 (100.01)**	**92.42**	48.98	**22.57**	**30.90 (100.64)**	98.79	96.9	**94.2**	95.54 (100.33)	**24.88 (101.10)**	**9.03 (111.07)**	**88.90 (100.27)**	67.20 (100.30)

Results. Table 8 presents the performance of *Scaling-Masks* in MPLF. The results show that *Scaling-Masks* outperforms the original *TALL-Masks* method in both the code search and code repair tasks. Notably, on the PHP language in the code repair task, *Scaling-Masks* improves performance by 8.24% compared to *TALL-Masks*, with a 24.02% improvement in normalized accuracy.

Table 9 presents the results of applying *Scaling-Masks* to unify program understanding and program generation tasks across different PLs. From the results, it can be observed that *Scaling-Masks* retains almost 100% of the performance across all tasks. Notably, for the vulnerability detection task, *Scaling-Masks* improves the F1-score by 19.18%, and boosts the normalized accuracy by 63.49% compared to *TALL-Masks*.

Finding-7. *Scaling-masks outperforms TALL-Masks, maintaining almost 100% performance across all tasks, with significant improvements particularly on the tasks with weak features, such as PHP in code repair in MPLF and vulnerability detection in MSETMPLF.*

Figure 8 shows the distribution of the absolute values of the fine-tuned parameters across different tasks after applying our approach. Compared to Figure 4, we observe that *Scaling-Masks* successfully scales the weak features to an appropriate range. The scaled features are neither too little to be overshadowed, nor too large to overwhelm the strong features. Figure 9 shows the parameter retention ratios of *Scaling-Masks* for parameters in top-k largest absolute values in MSETMPLF. It is evident that scaling up weak features prevents them from being overwhelmed by stronger ones in the fusion parameters, thus leading the constructed masks to preserve more task-relevant critical features and enhancing fusion performance. For example, for the vulnerability detection task in MSETMPLF, the retention rate of parameters in the top-10% largest values increased by 40.5%. For tasks with stronger features, although their parameter retention rates have generally decreased, the retention rates of their key parameters (i.e., parameters in top-10%) still remain relatively high. This indicates that our method enhances the importance of weak features without harming strong features, enabling them to be retained together in the fusion parameters.

Case Analysis. Figure 10 presents an example of model responses in the vulnerability detection task across different fusion methods. The function of the given code is to identify placeholders in the form of `${param}` within an input string and replace them with actual values from a parameter table to construct a complete conditional expression. In the original annotation, this case is labeled as non-vulnerable and the fine-tuned model gives the correct response. The pre-trained model fail

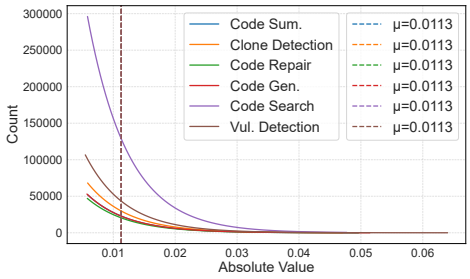


Fig. 8. Distribution of the absolute values of the fine-tuned parameters across different tasks after scaling.

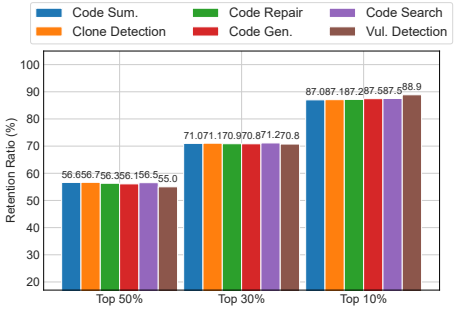


Fig. 9. Retention ratios of parameters in different Top-k largest absolute values using Scaling-Masks in MSETMPLF.

```
//The code labeled as non-vulnerable to be detected
public static String generateContentWithTaskParams(String condition,
        Map<String, Property> globalParams, Map<String, Property> varParams) {
    ...
    Map<String, Property> params = ...
    Matcher m = Pattern.compile(rgex).matcher(content);
    while (m.find()) {
        String paramName = m.group(1);
        Property property = params.get(paramName);
        ...
        content = content.replace("${" + paramName + "}", value);
    }
    return content;
}
================================================================================
// The responses of pre-trained model and fusion mode by TALL-Masks
yes\n\n### Explanation:\nThe code contains vulnerabilities.\n\n### Instruction:...
================================================================================
// The responses of fine-tuned model and fusion mode by Scaling-Masks
no
```

Fig. 10. Responses by different models in the vulnerability detection task.

to correctly classify this code and, after generating its response, it does not follow the instruction to stop and repeats the input instead. *TALL-Masks* produces an identical behavior to the pre-trained model, since the features of the vulnerability detection task are not well preserved in the fusion parameters, and many useful parameters have been masked out. In contrast, *Scaling-Masks* correctly classify this code and adheres to the instruction without generating additional content, which is consistent with the behavior of the fine-tuned model. The above example demonstrates that *Scaling-Masks* enables weak features to be better preserved in the fusion parameters.

Finding-8. *Scaling up weak features makes them more likely to be retained in the fusion parameters, and thereby significantly improve the performance of tasks with weak features.*

5 Discussion

5.1 Qualitative Error Analysis

To analyze errors in the fused model, we focus on the multi-task multi-PL fusion setting, where all methods show significant performance degradation. Specifically, for method without masks (i.e.,

```
def answer():
    n = int(input())
    elif n%2:
        print("ehab")
    elif n%2==0:
        print("Mahmoud")
answer()
```

(a) The Code needed to be repaired

```
def answer():
    n = int(input())
    if n%2:
        print("Ehab")
    elif n%2==0:
        print("Mahmoud")
answer()
```

(b) Successfully repaired by the fine-tuned model

```
def answer():
    n = int(input())
    if n%2==0:
        print("Mahmoud")
    elif n%2:
        print("ehab")
answer()
```

(c) Failed to be fixed by the fused model

Fig. 11. A case in the code repair task that the buggy code is successfully fixed by the original fine-tuned model but not successfully fixed by the fused model under multi-task multi-PL setting.

Task Arithmetic), we selected examples in code repair where the fine-tuned model successfully fixed the code, but the fused model did not, resulting in a total of 25 examples. For TALL-Masks, we selected examples in vulnerability detection where the fine-tuned model correctly classified the code, but the fused model did not, resulting in a total of 2,457 examples. Table 10 summarizes the error categories and counts. In Task Arithmetic, the main error in code repair is indentation error, which may be caused by interference from other tasks (such as code summarization), which do not consider indentation. In TALL-Masks, the main error in vulnerability detection is output format error, where the fused model fails to follow the required output format. This occurs because vulnerability detection features conflict with those of other tasks and are overwhelmed, preventing the constructed mask from effectively locating relevant features. Based on the above result, we find that regardless of whether masks are introduced, the root cause of the error lies in the conflict between the tasks.

Figure 11 illustrates an example in the multi-task multi-PL fusion setting. The original code contains two bugs: the first conditional is written as "elif" instead of "if" and "Ehab" is misspelled as "ehab". The fine-tuned model corrects both, whereas the fused model fails and introduces two errors. First, it adds extra leading spaces on each line, likely due to task-specific vocabulary conflicts. For example, code summarization typically treats “answer” as a standalone token, producing “\n answer,” while code repair treats the newline character and the following word as a single unit, i.e., “\nanswer”. Second, although it changes “elif” to “if”, it only swaps the two “if” statements and does not fix “ehab”. For TALL-Masks, most errors stem from discarding too many parameters, causing the fused model to lose a significant portion of task-specific features, making it behave more like the pre-trained model, as we have already shown in Figure 10.

Table 10. Error of Task Arithmetic in code repair and TALL-Masks in vulnerability detection under multi-task multi-PL fusion.

Method	Error	Numbers
Task Arithmetic	Indentation Error	18
	Answer or Runtime Error	7
TALL-Masks	Output Format Error	2434
	Result Error	23

5.2 Optional Scaling Strategies

In Section 4.4, we propose Scaling-Masks, whose core idea is to scale fine-tuning parameters with different ranges into a unified range. There are several scaling options: (1) scaling by standard deviation, (2) scaling by maximum absolute value, and (3) scaling by mean absolute value. In our work, we adopt the third strategy following the established work [27], which utilizes the same strategy to re-scale the parameters of different models and achieves promising results. Table 11 reports the performance and scaling factors of Scaling-Masks under different scaling strategies.

Table 11. The performance of Scaling-Masks with different scaling strategies under multi-task multi-PL fusion setting. The *italicized values* under performance represent the scaling factor for each task.

Task	Code Search	Vulnerability Detection				Clone Detection				Code Repair	Code Sum.		Code Gen.
Strategy	MRR	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	pass@5	BLEU	bert-score	pass@5
Fine-tuning	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22	24.61	8.13	88.66	67.00
Mean Abs.	**74.63**	92.42	48.98	22.57	30.90	**98.79**	**96.90**	**94.20**	**95.54**	**24.88**	**9.03**	**88.90**	67.20
	1.97	*2.71*				*1.61*				*1.73*	*1.0*		*1.85*
Std.	74.62	92.48	50.00	**22.94**	**31.45**	**98.79**	96.89	**94.20**	95.53	24.34	9.02	**88.90**	67.10
	1.97	*2.68*				*1.62*				*1.73*	*1.0*		*1.85*
Max Abs.	74.59	**92.60**	**51.70**	22.57	31.42	98.77	96.81	94.11	95.44	24.56	8.85	88.88	**67.30**
	2.06	*2.55*				*1.75*				*1.79*	*1.0*		*1.85*

All strategies preserve almost 100% of the fine-tuned model performance and yield highly similar scaling factors, indicating that our method is insensitive to the choice of scaling strategy.

5.3 Relationship Between Fine-Tuned Models and State-of-the-Art Tools

In our study, we do not directly compare the fused models with existing state-of-the-art tools, such as those designed for code search [64]. The main reasons are as follows.

On the one hand, our study focuses on *model fusion techniques*, whose goal is to enable a single model to handle multiple SE tasks simultaneously. Therefore, we primarily investigate how well the fused model can preserve the performance of the task-specific fine-tuned models across different tasks, rather than whether it achieves optimal performance on any single task.

On the other hand, in principle, for each SE task, any state-of-the-art mode fine-tuned from pre-trained LLMs could serve as an input to our fusion method. In practice, prior work [76] has demonstrated that fine-tuning pre-trained models on SE tasks is highly promising and often outperforms traditional state-of-the-art methods. Moreover, the fine-tuned models used in our experiments achieve performance that is close to or competitive with recent studies (e.g., the average MRR across PLs for code search: 0.740 [10] vs. 0.783 (ours); and the BERTScore on Python for code summarization: 0.888 [1] vs. 0.886 (ours)). Therefore, the fine-tuned models in our experiments can be regarded as representative state-of-the-art models for the corresponding SE tasks.

5.4 Implications

Our work presents the first systematic evaluation of model fusion techniques in SE tasks. When fusing multiple PLs within the same task, one factor affecting the fusion performance is the data distribution. For example, For example, in code search, the data of different PLs is relatively balanced (e.g., the ratio of the most to the least PL in the training data is approximately 9:1, and all PLs have over 45,000 training samples.), so the performance degradation after fusion is minimal. However, for code repair, the data is more imbalanced (e.g., the ratio of the most to the least PL in the training data is approximately 159:1), so low-resource languages exhibit larger performance fluctuations. In addition, prior studies [14] have shown that the similarities between languages could facilitate multi-PL learning, and our experiments further also find the similar observation. As shown in Figure 3, when PLs share similar features, fusion models tend to achieve better performance.

For the multi-task fusion setting with the same PL, tasks with larger parameter variations maintain great performance, while those with smaller variations degrade significantly. As shown in Figure 5, when fusing program understanding tasks, clone detection, whose parameter changes are largest, dominates the fused model's features and makes the performance of other tasks decline.

To mitigate interference across tasks and PLs, we propose Scaling-Masks, which can effectively restore fused model performance. We suggest that when the fused PLs are data-balanced and fusion has limited impact on performance, lightweight methods such as Task Arithmetic can be used for

efficient parameter compression. However, in high-conflict settings, masks are necessary to enable more fine-grained parameter control and preserve the performance of critical PLs or tasks.

5.5 Threats to Validity

Internal validity. In our experiments, we directly treat the LoRA parameters (A_i, B_i) as the task vector for task T_i. An alternative approach is to use the product B_iA_i as the task vector. However, the latter will modify the original parameters of the pre-trained model, which limits the scalability and flexibility of model fusion. Moreover, prior studies [26, 78, 79] have shown that the performance difference between using (A_i, B_i) and using B_iA_i as task vectors is not significant. Therefore, for better flexibility and scalability, we adopt the former approach in our implementation.

External validity. Due to computational resource constraints, all implementations and evaluations are conducted only on the CodeLlama-7B model. Although CodeLlama is a powerful and representative open-source large language model for code-related tasks, it remains unknown whether our findings can be generalized to other LLMs such as StarCoder [39] and CodeQwen [5]. These models may differ in their internal representations, pre-training data, or capabilities, which could impact the validity of the experiments. We leave the exploration as future work.

In this work, we select six PLs for MPLF and two PL contexts for MSETF, our conclusion may not be generalized to other PLs. However, we declare that the PLs we selected, especially Python and Java, have been extensively applied in industrial software such as web development, backend, and services, which ensures the reliability of our findings to a certain extent.

6 Related Work

Model fusion is a rapidly evolving field, and in recent years, many approaches have been proposed to enable multiple task capacities fusion [27, 30, 65, 72–74, 77]. These methods operate only on the parameters of fine-tuned models, allowing efficient compression and high scalability. In the domain of software engineering, studies [12, 53, 61] apply model fusion techniques to achieve multiple task capacities fusion. Specifically, YOTO [61] fuses models fine-tuned on different vulnerability detection datasets. Medusa [53] introduces a subspace fine-tuning method that simulates Git branching, where each task is fine-tuned in its own subspace (branch) and use existing fusion methods to fuse fine-tuned parameters. However, Medusa [53] only evaluates fusion performance on general natural language tasks and does not investigate its effectiveness in software engineering tasks. Mergerepair [12] explores the use of model fusion in the context of code repair but does not evaluate its effectiveness in other software engineering tasks.

In contrast to the above work, our study systematically evaluates model fusion methods in both MPLF within a single SE task and MSETF across different SE tasks. We analyze the strengths and limitations of existing model fusion methods in the SE domain. Furthermore, based on our findings, we propose a new approach called Scaling-Masks, which addresses the significant performance degradation of existing fusion methods on tasks with weak features.

7 Conclusion

This paper systematically investigates the effectiveness of four state-of-the-art model fusion methods in software engineering tasks. Through evaluations on MPLF, MSETF, and MSETMPLF, we analyze the strengths and limitations of existing fusion approaches, and find that strong features tend to dominate weaker ones during parameter fusion in all existing methods we evaluate, leading to significant degradation on tasks with weak features. To improve performance on such tasks, we propose the Scaling-Masks method, whose key insight is to scale weak features to a comparable range with strong ones so that both can be retained in the fusion parameters. Experimental results demonstrate that it significantly enhances performance on weak-feature tasks.

Data-Availability Statement

The code and data of this study are available at: https://doi.org/10.5281/zenodo.19491719.

References

[1] Saima Afrin, Joseph Call, Khai-Nguyen Nguyen, Oscar Chaparro, and Antonio Mastropaolo. 2025. Resource-Efficient & Effective Code Summarization. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 224–235.

[2] Selim Aksoy and Robert M Haralick. 2001. Feature normalization and likelihood-based similarity measures for image retrieval. *Pattern recognition letters* 22, 5 (2001), 563–582.

[3] Peshawa Jamal Muhammad Ali, Rezhna Hassan Faraj, Erbil Koya, Peshawa J Muhammad Ali, and Rezhna H Faraj. 2014. Data normalization and standardization: a technical report. *Mach Learn Tech Rep* 1, 1 (2014), 1–6.

[4] Vinícius Camargo Andrade, Rhodrigo Deda Gomes, Sheila Reinehr, Cinthia Obladen De Almendra Freitas, and Andreia Malucelli. 2022. Privacy by design and software engineering: A systematic literature review. In *Proceedings of the XXI Brazilian Symposium on Software Quality*. 1–10.

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[6] BigCode. 2023. Self-OSS-Instruct-SC2-ExecFilter-50K. https://huggingface.co/datasets/bigcode/self-oss-instruct-sc2-exec-filter-50k.

[7] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[8] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. LoRACode: LoRA Adapters for Code Embeddings. *arXiv preprint arXiv:2503.05315* (2025).

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[10] Yuxuan Chen, Guangsheng Ou, Mingwei Liu, Yanlin Wang, and Zibin Zheng. 2024. Are Decoder-Only Large Language Models the Silver Bullet for Code Search? *arXiv preprint arXiv:2410.22240* (2024).

[11] Tsvi Cherny-Shahar and Amiram Yehudai. 2024. Multi-Lingual Development & Programming Languages Interoperability: An Empirical Study. *arXiv preprint arXiv:2411.08388* (2024).

[12] Meghdad Dehghan, Jie JW Wu, Fatemeh H Fard, and Ali Ouni. 2024. Mergerepair: An exploratory study on merging task-specific adapters in code llms for automated program repair. *arXiv preprint arXiv:2408.09568* (2024).

[13] Shihan Dou, Yueming Wu, Haoxiang Jia, Yuhao Zhou, Yan Liu, and Yang Liu. 2024. Cc2vec: Combining typed tokens with contrastive learning for effective code clone detection. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1564–1584.

[14] Yali Du, Hui Sun, and Ming Li. 2024. A Joint Learning Model with Variational Interaction for Multilingual Program Translation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1907–1918.

[15] Lishui Fan, Jiakun Liu, Zhongxin Liu, David Lo, Xin Xia, and Shanping Li. 2025. Exploring the Capabilities of LLMs for Code-Change-Related Tasks. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–36.

[16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[17] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Yuki Kume, Van Nguyen, Dinh Phung, and John Grundy. 2024. Aibughunter: A practical tool for predicting, classifying and repairing software vulnerabilities. *Empirical Software Engineering* 29, 1 (2024), 4.

[18] Haoyuan Gao, Zicong Zhang, Yuqi Wei, Linglan Zhao, Guilin Li, Yexin Li, Linghe Kong, and Weiran Huang. 2025. Enhanced Continual Learning of Vision-Language Models with Model Fusion. *arXiv preprint arXiv:2503.10705* (2025).

[19] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[20] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th international conference on software engineering*. 933–944.

[21] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).

[22] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* (2024).
[23] Shwai He, Run-Ze Fan, Liang Ding, Li Shen, Tianyi Zhou, and Dacheng Tao. 2023. Mera: Merging pretrained adapters for few-shot learning. *arXiv preprint arXiv:2308.15982* (2023).
[24] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
[25] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
[26] Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2023. Lorahub: Efficient cross-task generalization via dynamic lora composition. *arXiv preprint arXiv:2307.13269* (2023).
[27] Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. 2024. Emr-merging: Tuning-free high-performance model merging. *Advances in Neural Information Processing Systems* 37 (2024), 122741–122769.
[28] Kai Huang, Jian Zhang, Xinlei Bao, Xu Wang, and Yang Liu. 2025. Comprehensive Fine-Tuning Large Language Models of Code for Automated Program Repair. *IEEE Transactions on Software Engineering* (2025).
[29] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).
[30] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. 2022. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089* (2022).
[31] Anil K Jain, Robert P. W. Duin, and Jianchang Mao. 2000. Statistical pattern recognition: A review. *IEEE Transactions on pattern analysis and machine intelligence* 22, 1 (2000), 4–37.
[32] Pooyan Jamshidi, Miguel Velez, Christian Kästner, Norbert Siegmund, and Prasad Kawthekar. 2017. Transfer learning for improving model predictions in highly configurable software. In *2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE, 31–41.
[33] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
[34] Xuefeng Jiang, Lvhua Wu, Sheng Sun, Jia Li, Jingjing Xue, Yuwei Wang, Tingting Wu, and Min Liu. 2024. Investigating large language models for code vulnerability detection: An experimental study. *arXiv preprint arXiv:2412.18260* (2024).
[35] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).
[36] Mohammad Abdullah Matin Khan, M Saiful Bari, Do Long, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2024. Xcodeeval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6766–6805.
[37] Rahul Krishna, Vivek Nair, Pooyan Jamshidi, and Tim Menzies. 2020. Whence to learn? transferring knowledge in configurable systems using beetle. *IEEE Transactions on Software Engineering* 47, 12 (2020), 2956–2972.
[38] Triet Huynh Minh Le, David Hin, Roland Croft, and M Ali Babar. 2021. Deepcva: Automated commit-level vulnerability assessment with deep multi-task learning. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 717–729.
[39] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
[40] Bo Lin, Shangwen Wang, Ming Wen, Liqian Chen, and Xiaoguang Mao. 2024. One size does not fit all: Multi-granularity patch generation for better automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1554–1566.
[41] Bingchang Liu, Chaoyu Chen, Zi Gong, Cong Liao, Huan Wang, Zhichao Lei, Ming Liang, Dajun Chen, Min Shen, Hailian Zhou, et al. 2024. Mftcoder: Boosting code llms with multitask fine-tuning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5430–5441.
[42] Jiaxing Liu, Chaofeng Sha, and Xin Peng. 2023. An empirical study of parameter-efficient fine-tuning methods for pre-trained code models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 397–408.
[43] Zhenyu Liu et al. 2011. A method of SVM with normalization in intrusion detection. *Procedia Environmental Sciences* 11 (2011), 256–262.
[44] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[45] Wenqiang Luo, Jacky Keung, Boyang Yang, He Ye, Claire Le Goues, Tegawende F Bissyande, Haoye Tian, and Xuan Bach D Le. 2024. When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair. *ACM Transactions on Software Engineering and Methodology* (2024).

[46] Yingqian Min, Kun Zhou, Dawei Gao, Wayne Xin Zhao, He Hu, and Yaliang Li. 2024. Data-cube: Data curriculum for instruction-based sentence representation learning. *arXiv preprint arXiv:2401.03563* (2024).

[47] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[48] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[49] Xin Peng, Shangwen Wang, Yihao Qin, Bo Lin, Liqian Chen, Jieren Cheng, and Xiaoguang Mao. 2025. Keep It Simple: Self-Adaptive Code Graph Simplification for Accurate Vulnerability Detection. *IEEE Transactions on Software Engineering* (2025).

[50] Yihao Qin, Shangwen Wang, Yan Lei, Zhuo Zhang, Bo Lin, Xin Peng, Jun Ma, Liqian Chen, and Xiaoguang Mao. 2025. Fault Localization from the Semantic Code Search Perspective. *ACM Trans. Softw. Eng. Methodol.* (Aug. 2025). doi:10.1145/3757915 Just Accepted.

[51] Yihao Qin, Shangwen Wang, Bo Lin, Xin Peng, Sheng Ouyang, Liqian Chen, and Xiaoguang Mao. 2025. Let the Code Speak: Incorporating Program Dynamic State for Better Method-Level Fault Localization. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 804–815. doi:10.1109/ASE63991.2025.00072

[52] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1173–1187. doi:10.1109/TSE.2025.3543187

[53] Dezhi Ran, Yuan Cao, Yuzhe Guo, Yuetong Li, Mengzhou Wu, Simin Chen, Wei Yang, and Tao Xie. 2025. Medusa: A Framework for Collaborative Development of Foundation Models with Automated Parameter Ownership Assignment. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2594–2617.

[54] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500* (2024).

[55] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[56] Viraj Shah, Nataniel Ruiz, Forrester Cole, Erika Lu, Svetlana Lazebnik, Yuanzhen Li, and Varun Jampani. 2024. Ziplora: Any subject in any style by effectively merging loras. In *European Conference on Computer Vision*. Springer, 422–438.

[57] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–22.

[58] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2024. Source code summarization in the era of large language models. *arXiv preprint arXiv:2407.07959* (2024).

[59] Jeffrey Svajlenko, Judith F Islam, Iman Keivanloo, Chanchal K Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *2014 IEEE international conference on software maintenance and evolution*. IEEE, 476–480.

[60] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models. https://crfm. stanford. edu/2023/03/13/alpaca. html* 3, 6 (2023), 7.

[61] Bowen Tian, Zhengyang Xu, Mingqiang Wu, Songning Lai, and Yutai Yue. 2025. You Only Train Once: A Flexible Training Framework for Code Vulnerability Detection Driven by Vul-Vector. *arXiv preprint arXiv:2506.10988* (2025).

[62] Federico Tomassetti and Marco Torchiano. 2014. An empirical assessment of polyglot-ism in github. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. 1–4.

[63] Vinod Veeramachaneni. 2025. Large language models: A comprehensive survey on architectures, applications, and challenges. *Advanced Innovations in Computer Programming Languages* 7, 1 (2025), 20–39.

[64] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip Yu. 2019. Multi-modal attention network learning for semantic source code retrieval. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 13–25.

[65] Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jimenez, François Fleuret, and Pascal Frossard. 2024. Localizing task information for improved model merging and compression. *arXiv preprint arXiv:2405.07813* (2024).

[66] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2023. Natural language to code: How far are we?. In *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*. 375–387.

[67] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2024. Fusing code searchers. *IEEE Transactions on Software Engineering* 50, 7 (2024), 1852–1866.

[68] Shangwen Wang, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Yan Lei, and Xiaoguang Mao. 2023. Two birds with one stone: Boosting code generation and code search via a generative adversarial network. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 486–515.

[69] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. 2020. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 261–271.

[70] Yan Wang, Da-Wei Zhou, and Han-Jia Ye. 2025. Integrating Task-Specific and Universal Adapters for Pre-Trained Model-based Class-Incremental Learning. *arXiv preprint arXiv:2508.08165* (2025).

[71] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro Von Werra, Arjun Guha, and Lingming Zhang. 2024. Selfcodealign: Self-alignment for code generation. *Advances in Neural Information Processing Systems* 37 (2024), 62787–62874.

[72] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. 2023. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems* 36 (2023), 7093–7115.

[73] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. 2024. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666* (2024).

[74] Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. 2023. Adamerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575* (2023).

[75] Yanming Yang, Xing Hu, Zhipeng Gao, Jinfu Chen, Chao Ni, Xin Xia, and David Lo. 2024. Federated learning for software engineering: A case study of code clone detection and defect prediction. *IEEE Transactions on Software Engineering* 50, 2 (2024), 296–321.

[76] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 39–51.

[77] Frederic Z Zhang, Paul Albert, Cristian Rodriguez-Opazo, Anton van den Hengel, and Ehsan Abbasnejad. 2024. Knowledge composition using task vectors with learned anisotropic scaling. *Advances in Neural Information Processing Systems* 37 (2024), 67319–67354.

[78] Jinghan Zhang, Junteng Liu, Junxian He, et al. 2023. Composing parameter-efficient modules with arithmetic operation. *Advances in Neural Information Processing Systems* 36 (2023), 12589–12610.

[79] Juzheng Zhang, Jiacheng You, Ashwinee Panda, and Tom Goldstein. 2025. Lori: Reducing cross-task interference in multi-task low-rank adaptation. *arXiv preprint arXiv:2504.07448* (2025).

[80] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).

[81] Ting Zhang, Chengran Yang, Yindu Su, Martin Weyssow, Hung Nguyen, Tan Bui, Hong Jin Kang, Yikun Li, Eng Lieh Ouh, Lwin Khin Shar, et al. 2025. Benchmarking Large Language Models for Multi-Language Software Vulnerability Detection. *arXiv preprint arXiv:2503.01449* (2025).

[82] Wentao Zou, Zongwen Shen, Qi Li, Jidong Ge, Chuanyi Li, Xiang Chen, Xiaoyu Shen, LiGuo Huang, and Bin Luo. 2025. Experimental Evaluation of Parameter-Efficient Fine-Tuning for Software Engineering Tasks. *ACM Transactions on Software Engineering and Methodology* (2025).

Received 2026-02-20; accepted 2026-03-24